

Python Libraries used:

1. numpy
2. pandas
3. matplotlib.pyplot
4. sklearn.linear_model
5. sklearn.metrics
6. Xgboost

Workflow:

1. **Data Ingestion** : The dataset was loaded into the jupyter notebook using pandas.
2. We did all EDA on the raw, uncleaned data first. The point was to understand what the data looks like before touching it — so that every cleaning decision came from an actual observation, not a guess.
3. **Data Imputation** : (All imputations were done using training dataset. To impute the testing dataset, we used the values and ideas inferred from the training dataset. This was done in order to prevent leakage)
 - a. *Missing values:*
 - I. First thing we checked was how much data is missing and where.
 - II. Transmission was the biggest gap(11.7%) . But when we look at the distribution, it's 380,000 automatic vs 14,000 manual — so anything missing is almost certainly automatic.
 - III. Two things came out of looking at ‘condition’ carefully. First, the relationship between condition and price is non-linear. From condition 5 down to about 2, price drops gradually and fairly predictably. But below 2, price falls off a cliff. Second, the median condition varies a lot by year. Hence, the missing condition was imputed with PER-YEAR median, not global median. Global median (3.6) would badly over-estimate condition for 2001–2006 cars and under-estimate for 2013–2015 cars
 - IV. Missing values of the feature odometer (which had a missing percentage of 0.01%), were imputed using median odometer value.
 - V. In all the rest categorical features, i.e colour, interior, body, make, etc, were imputed by replacing NaNs with “Unknown”.
 - b. *Outliers:*
 - I. In the feature ‘Selling Price’, prices ranged from \$1 to \$230,000. The distribution is heavily right-skewed —most cars sit between \$5k and \$25k, with a long tail of expensive outliers.

Two groups of outliers stood out:

Below \$500:

About 8,400 rows. These are salvage titles, insurance write-offs, and plain data entry errors (a car listed at \$1 isn't being sold for \$1). They don't behave like normal wholesale cars and would confuse the model. Hence, \$500 was used as the lower fence

Above ~\$52,000:

Roughly 3,200 rows — exotics and ultra-luxury cars. A Ferrari and a Ford F-150 don't depreciate the same way. Including them would distort the model's understanding of the mainstream market. We used the $3 \times \text{IQR}$ rule for the upper fence rather than the more common $1.5 \times \text{IQR}$, because $1.5 \times$ was cutting into legitimate trucks and loaded SUVs that genuinely sell for \$35k–\$45k at wholesale. $3 \times \text{IQR}$ only removes the true extremes.

II. The odometer column had a long tail of obvious data errors. Values like 999,999 miles are physically impossible for consumer cars. The 99th percentile was around 226,000 miles — anything above that is basically guaranteed to be a typo or a system default value.

We decided to cap rather than drop those rows, because even if the odometer is wrong, the rest of the row, i.e. make, year, condition, is still good training data.

- c. Title case normalization was done for the “body” feature, because both “Sedan” and “sedan” existed in the dataframe.
- d. Also, the values in model/trim having frequency less than 0.1% were grouped into “Other”. This was done to smoothen the dataset.

4. Feature Engineering :

Features created are

- a. $\text{car_age} = \text{current_year} - \text{year}$
- b. $\text{usage_intensity} = \text{odometer} / \text{car_age}$
- c. $\text{condition_age_ratio} = \text{condition} / \text{car_age}$
- d. $\text{log_odometer} = \log(1 + \text{odometer})$

5. Encoding :

- a. We used Target Encoding to encode the features. In this, we replaced each category with the mean selling price of that category in the training set. This is an immediately useful signal. The model can directly compare these numbers without any intermediate learning step.
- b. We didn't use Label Encoding because it turns categories into integers which mean nothing. The model has to waste capacity learning that the ordering
- c. If we compute these means on the full training set and then train on it, row's own price is contributing to its own encoding. That's circular. The model sees a version of the answer during training and overfits. To fix that, we used a 5-fold out-of-fold encoding. We split training data into 5 folds and for each fold, we computed category means from the other 4 folds only. We used those means to encode the held-out fold. We didn't use the direct mean though, we used the smoothed mean. Smoothing prevents rare categories from getting unreliable target-encoding values based on very few samples. It pulls their mean closer to the global average, reducing overfitting and making the encoding more stable during inference.

6. Model Training and Evaluation:

- a. We trained a baseline XGBRegressor model using default parameters and evaluated its performance using RMSE.
- b. We then applied RandomizedSearchCV to tune the model efficiently without performing an expensive full grid search. We tuned important XGBoost hyperparameters such as:
 - I. n_estimators
 - II. max_depth
 - III. learning_rate
 - IV. subsample
 - V. colsample_bytree
 - VI. reg_alpha
 - VII. reg_lambda
- c. We used RMSE as an evaluation metric of our model.

BIDDING SEQUENCE DESIGN:

1. Core Formulas:

- a. $\text{bankroll_ratio} = \text{current_bankroll} / \text{INITIAL_BANKROLL}$
- b. $\text{margin} = \text{BASE_MARGIN} \times (\text{bankroll_ratio} ^ 0.5)$
- c. $\text{max_bid} = \text{predicted_value} \times (1 - \text{margin})$
(capped at $\text{MAX_SINGLE_SPEND_PCT} \times \text{bankroll}$)
- d. $\text{increment} = \max(\text{MIN_INCREMENT}, \text{current_highest_bid} \times \text{INCREMENT_PCT})$
- e. $\text{next_bid} = \text{current_highest_bid} + \text{increment}$

2. The agent receives one car at a time as a plain Python dict. No pandas, no DataFrames, no operations across multiple rows. All the cleaning statistics (odometer cap, condition medians, target encoding maps, rare category sets) are loaded from the pkl file at startup and stored as instance variables. Preprocessing is then just arithmetic and dictionary lookups.

3. The feature vector is assembled in the exact same column order as training. This is enforced by storing `feature_cols` in the pkl and iterating over it.

4. Guards conditions (checked in order before every bid):

1. No prediction available
2. Bankroll critically low
3. Current bid already below minimum profit margin
4. Current bid already at or above our ceiling
5. Next bid would exceed ceiling -> bid at ceiling instead
6. Next bid exceeds available bankroll

5. Bidding logic:

1. We predict the car's value using our model. From that, we calculate the maximum we're willing to pay — which is predicted value minus a 12% profit margin. That margin isn't fixed though; it shrinks gradually as our bankroll depletes, so we naturally become more competitive when the budget gets low.
2. Each round, we bid just above the current highest bid by a small increment (1% of current bid, minimum \$200). We keep doing this until either we win or the next bid would exceed our ceiling — at which point we walk away.
3. We never spend more than 20% of remaining bankroll on a single car, and we stop bidding entirely if bankroll drops below 5% of the starting amount.
4. No round counter is used. Every bid decision is just: given this car's predicted value and our current financial state.

